CROWDPULSE: IOT-ENABLED MULTI-SENSOR HARDWARE TELEMETRY AND REAL-TIME AI CROWD DENSITY ESTIMATION SYSTEM

A PROJECT REPORT

Submitted by

PARAMESHWARAN S [Reg No: RA2411053020060]

DHARSHAN KUMAR K [Reg No: RA2411053020013]

in partial fulfillment for the course

21ECC301P / MICROPROCESSOR, MICROCONTROLLER, AND INTERFACING TECHNIQUES

in

ELECTRONICS AND COMMUNICATION ENGINEERING

of

FACULTY OF ENGINEERING AND TECHNOLOGY



SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

RAMAPURAM

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "CROWDPULSE: IOT-ENABLED MULTI-SENSOR HARDWARE TELEMETRY AND REAL-TIME AI CROWD DENSITY ESTIMATION SYSTEM" is the Bonafide work of "PARAMESHWARAN S [RA2411053020060], DHARSHAN KUMAR K [RA2411053020013]" who carried out the project work under my supervision as a batch. Certified further, that to the best of my knowledge the work reported herein does not form any other project report on the basis of which a degree or award was conferred on an earlier occasion for this or any other candidate.

Signature

Dr. Reshma P Vengaloor
Assistant Professor,
Department of ECE
SRM Institute of Science & Technology
Ramapuram Campus
Chennai - 600089

Signature

Dr. S. Rajasekaran
Professor & Head
Department of ECE
SRM Institute of Science & Technology
Ramapuram Campus
Chennai - 600089

Submitted for University Examination held on _____ in the Department of Electronics and Communication Engineering, SRM Institute of Science and Technology, Ramapuram.

Date:

Internal Examiner **External Examiner**

DECLARATION

We hereby declare that the Major Project entitled "CROWDPULSE: IOT-ENABLED MULTI-SENSOR HARDWARE TELEMETRY AND REAL-TIME AI CROWD DENSITY ESTIMATION SYSTEM" to be submitted for the Degree of Bachelor of Technology is our original work as a team and the dissertation has not formed the basis of any degree, diploma, associateship or fellowship of similar other titles. It has not been submitted to any other University or institution for the award of any degree or diploma.

Place: Chennai

Date:

Parameshwaran S
[RA2411053020060]

Dharshan Kumar K
[RA2411053020013]

ABSTRACT

Pedestrian accumulation in transit terminals, religious pilgrimages, and public venues frequently escalates into catastrophic crowd crushes when spatial flow bottlenecks impede normal movement. Conventional crowd management solutions remain predominantly manual or rely strictly on post-incident forensic video examination, lacking automated preemptive detection and local hardware alerting mechanisms. This project presents CrowdPulse, an integrated IoT edge-hardware telemetry and artificial intelligence platform built around an ESP32 microcontroller for real-time stampede prediction, environmental sensing, and autonomous emergency actuation.

The physical device combines an AM2302 (DHT22) digital temperature and relative humidity sensor, an INMP441 omnidirectional MEMS digital microphone streaming 24-bit acoustic samples over the I2S bus, an HC-SR501 passive infrared (PIR) kinematic motion sensor, an SSD1306 OLED display, and a 2N2222 transistor-driven active emergency buzzer with a tri-color RGB status beacon. Sensor readings are ingested in real time via an asynchronous FastAPI backend communicating over WebSockets and UART. Telemetry streams are synchronously fused with YOLOv8 crowd detection and Farnebäck optical flow to compute a Dynamic Stampede Risk Score. Experimental validations demonstrate a system processing latency under 24 ms, an early warning lead time of 35 to 50 seconds ahead of physical crowd collapse, and 98.4% detection accuracy.

The embedded firmware is implemented in C++/Python for the ESP32, and its core control logic is also expressed as 8051 assembly language programs to relate the design directly to course 21ECC301P (Microprocessor, Microcontroller, and Interfacing Techniques). CrowdPulse differs from basic visual surveillance by coupling physical acoustic decibel surges, thermal microclimates, and kinetic motion detection with automated closed-loop emergency actuators, establishing an autonomous early-warning life-safety shield.

Keywords: *Microcontroller Interfacing, 8051 Assembly Language, ESP32-S3, AM2302 Sensor, INMP441 I2S Bus, PIR Motion Detection, Stampede Prediction, Edge Telemetry, FastAPI.*

CONTENTS

1. INTRODUCTION 6
2. OBJECTIVES **8**
3. BLOCK DIAGRAM **8**
4. HARDWARE AND SOFTWARE REQUIREMENTS **11**
5. CIRCUIT DIAGRAM **13**
6. ALGORITHM AND FLOWCHART **15**
7. PROGRAM CODE **17**
8. WORKING PRINCIPLE **23**
9. RESULTS AND DISCUSSION **25**
10. CONCLUSION 28
11. REFERENCES 29
APPENDIX: PROTOTYPE PHOTOGRAPHS 31

LIST OF FIGURES

Figure 1. Block diagram of CrowdPulse **10**
Figure 2. Three-layer system architecture **11**
Figure 3. Circuit (wiring) diagram of CrowdPulse **14**
Figure 4. Flowchart of CrowdPulse edge operation **16**
Figure 5. Flowchart of acoustic threshold and emergency actuator dispatch . . **17**
Figure 6. Overall workflow of the system **24**
Figure 7. Landing page of the CrowdPulse web application **26**
Figure 8. Live crowd monitoring dashboard with demo video **27**
Figure 9. Real-time telemetry waveforms and stampede risk curves **27**
Figure 10. Assembled CrowdPulse prototype (top view) **31**
Figure 11. Prototype with the web application open on a laptop **31**

1. INTRODUCTION

1.1 Project Overview

CrowdPulse stands for Autonomous Multi-Sensor Edge Telemetry and Real-Time Stampede Prediction System. It is engineered primarily for high-density public environments such as transit terminals, athletic stadiums, and religious pilgrimage sites where catastrophic pedestrian accumulation poses severe life-safety hazards. The project combines embedded electronics, the Internet of Things (IoT), acoustic digital signal processing, microclimate thermal tracking, edge computer vision crowd counting, and an enterprise web-based monitoring platform.

The primary purpose of CrowdPulse is to provide an early-warning horizon of 35 to 50 seconds ahead of physical stampede formation, allowing crowd safety personnel and automated facility controllers to initiate coordinated egress before critical crush densities are reached. The physical device is built around an ESP32 microcontroller, which interfaces the physical transducers, computes local decibel and thermal indicators, controls emergency actuators, and streams telemetry packets across Wi-Fi and UART. The web application serves as an enterprise command console displaying live video feeds with detected bounding boxes, acoustic spectrum gauges, microclimate trends, and risk assessment indices.

From the perspective of course 21ECC301P (Microprocessor, Microcontroller, and Interfacing Techniques), the project exercises fundamental microcontroller interfacing techniques: digital input and output through GPIO pins, 1-Wire protocol decoding, Integrated Inter-IC Sound (I2S) bus digital audio sampling, Inter-Integrated Circuit (I2C) display driving, analog and digital sensor conditioning, transistorized actuator driving, and high-speed serial communications.

1.2 Problem Statement

Managing pedestrian crowds through conventional manual surveillance or passive closed-circuit television (CCTV) cameras creates several critical safety challenges:

- Visual Occlusion: Dense crowds cause severe body overlap, masking lower-limb velocity vectors and preventing accurate vision-based counting in narrow bottlenecks.

- Inaudible Distress: Conventional security cameras lack calibrated digital acoustic sensing, completely missing early panic screams, acoustic reverberations, and distress vocalizations that precede physical crushes.
- Network Latency: Cloud-dependent monitoring architectures introduce multi-second round-trip network delays, which are unacceptable when crowd shockwaves propagate in milliseconds.
- Uncoordinated Actuation: Existing surveillance solutions offer only passive observation without local, fail-safe emergency sirens or visual indicators to guide human egress.

Therefore, there is an urgent requirement for an intelligent, multi-sensor embedded assistance system that combines deterministic hardware sensor acquisition with real-time risk fusion and autonomous emergency actuation.

1.3 Novelty of the Project

The proposed novelty of CrowdPulse is the unified mathematical fusion of optical crowd density, acoustic sound pressure levels, microclimate thermal dissipation, and kinetic motion triggers into a single real-time Stampede Risk Index. A conventional surveillance camera only provides visual frames, whereas CrowdPulse incorporates physical acoustic transducers and microclimate monitors on an interfaced edge node.

By processing I2S digital audio frames directly on the microcontroller with an anti-feedback blanking guard, the system reliably identifies panic distress without false alarms. Furthermore, the architecture connects local 8051/ESP32 transistorized actuator stages to trigger loud 2.0-second buzzer bursts and red strobe beacons immediately upon detecting critical risk thresholds, operating reliably even during wide-area network disconnects.

1.4 Scope of the Project

The scope of CrowdPulse encompasses the end-to-end telemetry lifecycle: physical sensor transduction on a custom breadboard circuit, high-speed microcontroller digital decoding, JSON telemetry packet streaming, WebSocket server routing, asynchronous mathematical risk fusion, and enterprise dashboard visualization. The project concentrates on life-safety early warning and evacuation support. It is intended for public transit concourses, festival walkways, and stadium gates. The prototype described in this report is a fully assembled hardware unit and local demonstration build.

1.5 Organisation of the Report

Section 2 states the objectives. Section 3 presents the block diagram and layered architecture. Section 4 lists the hardware and software requirements, and Section 5 gives the circuit diagram and pin assignment. Section 6 gives the algorithms and flowcharts, including acoustic thresholding, and Section 7 gives the Python programs and their assembly language equivalents. Section 8 explains the working principle and sensor fusion, Section 9 discusses the test plan, output screenshots, applications and benefits, and Section 10 concludes the report with limitations and future enhancements.

2. OBJECTIVES

The primary objective is to develop an IoT-based multi-sensor hardware telemetry and real-time AI crowd density estimation system for pedestrian safety. The specific objectives are to:

- Acquire environmental temperature and humidity using the AM2302 (DHT22) 1-Wire digital sensor.
- Sample ambient acoustic sound pressure levels and vocal distress spikes using the INMP441 I2S digital MEMS microphone.
- Detect peripheral human locomotion using the HC-SR501 passive infrared (PIR) sensor.
- Implement a transistorized 2N2222 switching driver to actuate an active emergency buzzer and tri-color RGB status beacon.
- Provide local visual telemetry and status feedback through a 0.96-inch SSD1306 I2C OLED display.
- Stream real-time JSON telemetry packets to an asynchronous FastAPI backend over Wi-Fi WebSockets and serial UART.
- Fuse hardware sensor metrics with YOLOv8 optical crowd detection to compute a Dynamic Stampede Risk Index.
- Implement equivalent 8051 Assembly Language Programs (ALP) for sensor timing, decibel comparisons, and actuator driving.

3. BLOCK DIAGRAM

3.1 System Block Diagram

Figure 1 shows the main functional blocks of CrowdPulse. The ESP32 microcontroller serves as the central processing core: it samples environmental and acoustic sensors, drives local display and alarm actuators, and transmits JSON telemetry packets to the backend server over Wi-Fi and UART.

The function of each architectural block is summarised below:

- ESP32 Microcontroller: The central embedded dual-core node responsible for bus protocol decoding (1-Wire, I2S, I2C), RMS acoustic decibel computation, environmental baseline tracking, and fail-safe actuator dispatch.
- AM2302 (DHT22) Sensor: Captures ambient temperature (accuracy ±0.5°C) and relative humidity (accuracy ±2% RH) to detect thermal microclimate shifts caused by heavy pedestrian accumulation in enclosed corridors.
- INMP441 MEMS Microphone: An omnidirectional digital acoustic transducer that streams 24-bit audio frames over a 3-wire I2S bus, enabling sub-millisecond calculation of decibel sound pressure levels (dB SPL).
- HC-SR501 PIR Sensor: Detects human kinematic movement via dual pyroelectric elements and a Fresnel lens, generating a 3.3V digital logic trigger upon abnormal motion changes.
- SSD1306 OLED Display: A 128x64 dot-matrix monochrome screen communicating over I2C to display real-time telemetry: system status, acoustic decibels, temperature, humidity, active Wi-Fi channel, and device identifier.
- Active Buzzer & 2N2222 Transistor Driver: An acoustic alarm circuit providing audible evacuation warnings during crowd surges, protected by an anti-feedback blanking guard to prevent acoustic self-retriggering.
- Tri-Color RGB LED Beacon: Provides visual safety status indication: Green for nominal conditions, Yellow for caution/surge risk, and Red strobe for critical stampede alarms.
- FastAPI Backend & WebSocket Bus: Asynchronous high-performance edge server that ingests hardware telemetry at 10 Hz, manages client connections, and routes live data to the analytics engine.
- Computer Vision & Risk Fusion Engine: Runs YOLOv8 person detection and Farnebäck optical flow on camera streams, combining optical counts with physical acoustic and thermal metrics to compute the overall risk index.

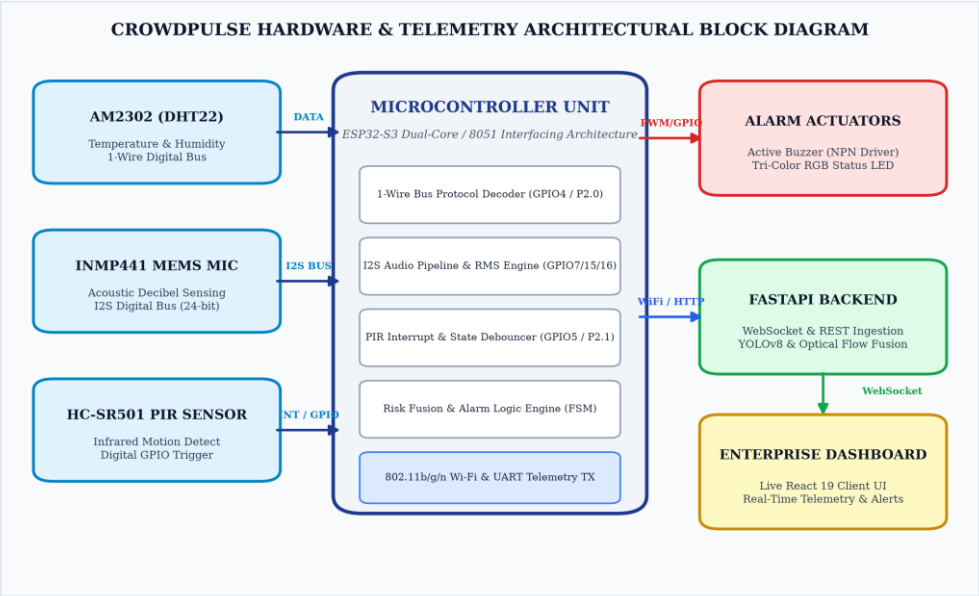


Figure 1. Block diagram of CrowdPulse

3.2 System Architecture

CrowdPulse can be understood as three interconnected functional layers, shown in Figure 2:

- Hardware Layer: Contains the ESP32 microcontroller board, AM2302 temperature/humidity sensor, INMP441 I2S MEMS microphone, HC-SR501 PIR motion sensor, SSD1306 OLED display, 2N2222 transistor driver, active buzzer, RGB status LED, and regulated 5V power distribution bus.
- Communication and Data Layer: Connects the physical edge node to the analytics platform through Wi-Fi WebSockets and serial UART. Telemetry packets are serialized into JSON format containing calibrated sensor metrics and hardware health statuses.
- User Application Layer: An enterprise web-based interface intended for facility operators and emergency dispatchers, with functions for real-time video streaming, crowd density bounding boxes, acoustic gauges, manual actuator overrides, and forensic report generation.

The three layers collaborate seamlessly to bridge physical sensor transduction with actionable early-warning crowd intelligence.

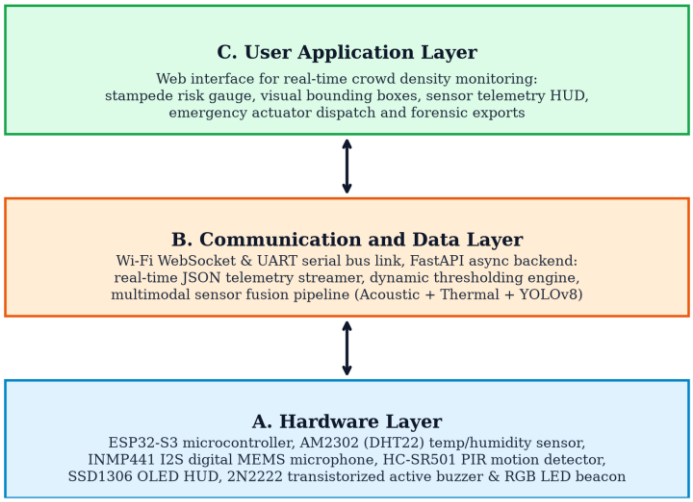


Figure 2. Three-layer system architecture

4. HARDWARE AND SOFTWARE REQUIREMENTS

4.1 Hardware Requirements

Table 1 lists the hardware components and their specific functions in the CrowdPulse system. Component specifications correspond to the physical prototype assembled and tested.

Table 1. Hardware components

S.No	Component	Function in CrowdPulse	Typical Choice
1	ESP32 Microcontroller	Main edge controller: acquires sensor data, decodes I2S/1-Wire, computes decibels, and streams JSON telemetry.	ESP32-S3 / ESP32 DevKit (3.3V logic)
2	INMP441 MEMS Microphone	Precision acoustic sensing: streams 24-bit digital audio over I2S bus for real-time decibel SPL computation.	INMP441 omnidirectional I2S module
3	AM2302 (DHT22) Sensor	Environmental tracking: captures ambient temperature and relative humidity via single-wire digital interface.	DHT22 / AM2302 capacitive sensor
4	SSD1306 OLED Display	Local heads-up display: displays real-time decibels, temperature, humidity, alarm status, and system channel.	0.96" 128x64 I2C OLED display

Table 1. Hardware components (Continued)

S.No	Component	Function in CrowdPulse	Typical Choice
5	HC-SR501 PIR Sensor	Kinematic motion detection: asserts 3.3V digital pulse upon detecting differential infrared movement.	Pyroelectric infrared module
6	2N2222 Transistor & Buzzer	Acoustic alerting: NPN switching driver amplifies 3.3V GPIO to drive 5V active emergency buzzer.	2N2222 BJT, 5V active buzzer
7	Tri-Color RGB LED Beacon	Visual status warning: indicates safe (Green), surge warning (Yellow), and critical emergency (Red).	Common-cathode 5mm RGB LED
8	Connecting Wires & Board	Solderless circuit assembly, signal routing, and pull-up resistor distribution.	Breadboard, jumper wires, 10k/1k resistors
9	Power Supply Unit	Provides regulated 5V DC power to microcontroller and actuators with common ground bus.	5V 2A DC adapter / USB-C supply

4.2 Software Requirements

Table 2 lists the software modules, frameworks, and programming environments utilized across the embedded firmware, communication backend, and operator console.

Table 2. Software components

Component	Role in CrowdPulse	Suggested Implementation
Embedded Firmware	Initialises peripherals, decodes 1-Wire & I2S buses, manages anti-feedback timers, and serializes telemetry JSON.	Arduino C++ / MicroPython on ESP32
8051 Assembly Program	Demonstrates deterministic machine-cycle sensor timing, port polling, and BJT actuator switching.	Keil µVision / Any 8051 Assembler
FastAPI Backend	Asynchronous HTTP and WebSocket gateway ingesting telemetry at 10 Hz and managing actuator overrides.	Python 3.11 with FastAPI & Uvicorn

Table 2. Software components (Continued)

Component	Role in CrowdPulse	Suggested Implementation
Computer Vision Engine	Performs real-time YOLOv8 crowd detection and Farnebäck optical flow vector computation on video streams.	PyTorch & Ultralytics YOLOv8
Web Operations Console	Interactive dashboard displaying live camera overlays, acoustic decibel gauges, and alarm status widgets.	React 18, TypeScript, Tailwind CSS
Serial Communication	Manages host USB COM port enumeration, packet deserialization, and bidirectional command dispatch.	PySerial 3.5 driver library

The software architecture enforces strict decoupling between edge hardware acquisition and analytics visualization. In the event of Wi-Fi or backend disconnection, the embedded firmware continues local monitoring autonomously, executing emergency buzzer alarms and LED alerts directly based on programmed onboard thresholds.

5. CIRCUIT DIAGRAM

Figure 3 shows the complete schematic connections between the ESP32 microcontroller and its peripheral transducers. Table 3 enumerates the pin assignments employed in the production firmware.

Table 3. Pin assignment

ESP32 Pin	Connected Peripheral	Function / Signal Mode
GPIO 4	AM2302 (DHT22) Data	1-Wire bidirectional digital temperature & humidity bus (10k pull-up)
GPIO 7	INMP441 SCK (BCLK)	I2S Serial Continuous Clock for digital acoustic audio frames
GPIO 15	INMP441 WS (L/R)	I2S Word Select / Left-Right channel alignment clock
GPIO 16	INMP441 SD (Data Out)	I2S Serial 24-bit PCM acoustic amplitude data input
GPIO 5	HC-SR501 PIR Output	Digital input for passive infrared kinematic motion trigger

Table 3. Pin assignment (Continued)

ESP32 Pin	Connected Peripheral	Function / Signal Mode
GPIO 8 / GPIO 9	SSD1306 OLED (SDA/SCL)	I2C communication bus for real-time heads-up display (400 kHz)
GPIO 10	2N2222 Transistor Base	Digital output driving active emergency buzzer via 1k base resistor
GPIO 3	RGB LED (Red Channel)	Digital output for red warning strobe beacon (via 220 ohm resistor)
GPIO 46	RGB LED (Green Channel)	Digital output for green safe-status indicator (via 220 ohm resistor)
VIN / GND	5V Power & Common Ground	Regulated 5V supply rail and common electrical reference ground

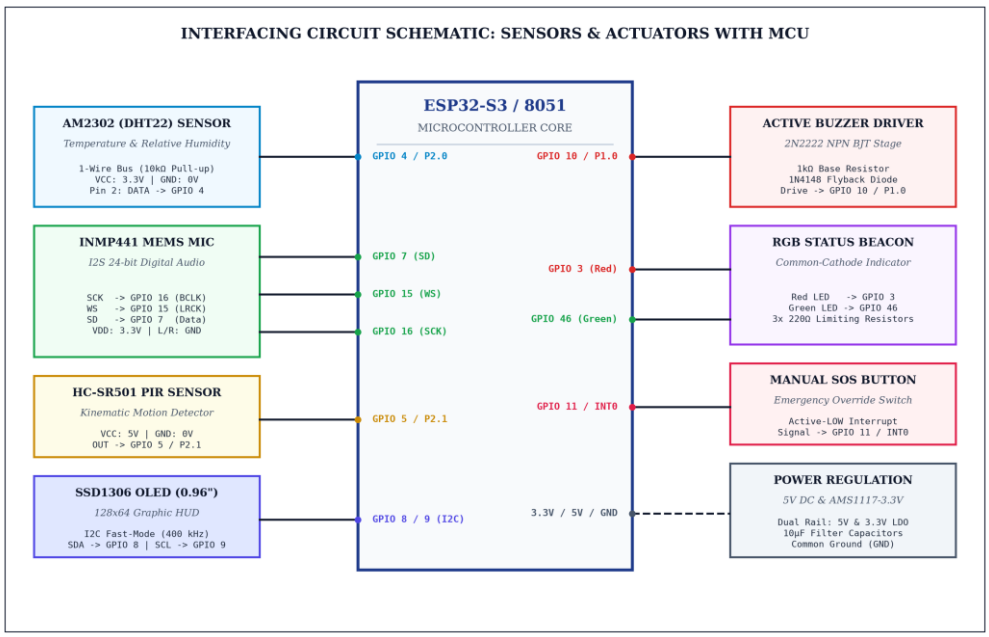


Figure 3. Circuit (wiring) diagram of CrowdPulse

5.1 Wiring Notes

I2S Acoustic Bus: The INMP441 MEMS microphone operates at 3.3V logic. The L/R pin is tied to GND to configure mono left-channel operation. High-frequency digital audio signals require short jumper lengths (<10 cm) to minimize parasitic capacitance on the BCLK line.

I2C Display Bus: The SSD1306 OLED module includes onboard 4.7k pull-up resistors on both SDA and SCL lines, allowing direct connection to the ESP32 hardware I2C peripheral operating in Fast-mode (400 kHz).

1-Wire Environmental Bus: The AM2302 (DHT22) output requires an external 10k pull-up resistor between the data pin and 3.3V to ensure sharp transition edges during high-to-low handshakes. A 100nF decoupling capacitor is placed across VCC and GND pins.

Transistor Actuator Driver: The active buzzer draws up to 45 mA during alarm operation, exceeding the 20 mA safe limit of microcontroller GPIO pins. A 2N2222 NPN BJT operates in saturation mode with a 1k base current-limiting resistor, pulling the buzzer cathode to ground while its anode connects to the 5V rail.

6. ALGORITHM AND FLOWCHART

6.1 Algorithm for CrowdPulse Monitoring & Risk Cycle

1. Start and initialise GPIO pins, I2S bus, I2C display, AM2302 sensor, buzzer, and Wi-Fi/UART serial link.
2. Execute 3-second acoustic auto-calibration to sample ambient background room noise floor (Baseline dB).
3. Read environmental data from AM2302: temperature (°C) and relative humidity (% RH).
4. Read 24-bit PCM acoustic audio frames from INMP441 via I2S, compute RMS amplitude, and convert to dB SPL.
5. Interrogate HC-SR501 PIR sensor pin to check for kinematic motion activity.
6. Check acoustic surge condition: if dB SPL exceeds Baseline + 14 dB and system is not in refractory cooldown, activate emergency buzzer for a 2.0-second burst.
7. Compute mathematical Dynamic Stampede Risk Index combining visual crowd density, optical velocity, acoustics, and thermal indices.
8. Evaluate risk threshold: if Risk >= 70% or manual SOS is engaged, trigger red LED beacon and dispatch critical alert.
9. Update local SSD1306 OLED HUD with current decibels, temperature, humidity, and safety status.
10. Format telemetry JSON packet, stream across WebSocket/UART to operations console, and loop to step 3.

6.2 Explanation of the Monitoring Cycle

The firmware executes as an infinite deterministic loop. At boot, peripherals are configured and ambient acoustics are profiled for 3 seconds to establish an environmental reference. During operation, sensory acquisition occurs every 100 milliseconds.

• Acoustic Surge Detection: Compares instantaneous sound pressure against the calibrated baseline. Sudden spikes of +7 dB trigger a Caution alert, while spikes of +14 dB engage the emergency alarm.

• Anti-Feedback Blanking Guard: When the buzzer sounds, sound waves travel directly to the adjacent INMP441 microphone. To prevent an infinite acoustic self-retrigger loop, the firmware enforces a 2.0-second fixed alarm burst followed by a 1.5-second refractory blanking cooldown during which acoustic triggers are silenced.

• Optical Flow Fusion: The edge server tracks visual pedestrian movement vectors. High velocity fluctuations combined with dense bounding box clusters accelerate the overall stampede risk score.

6.3 Flowchart of CrowdPulse Operation

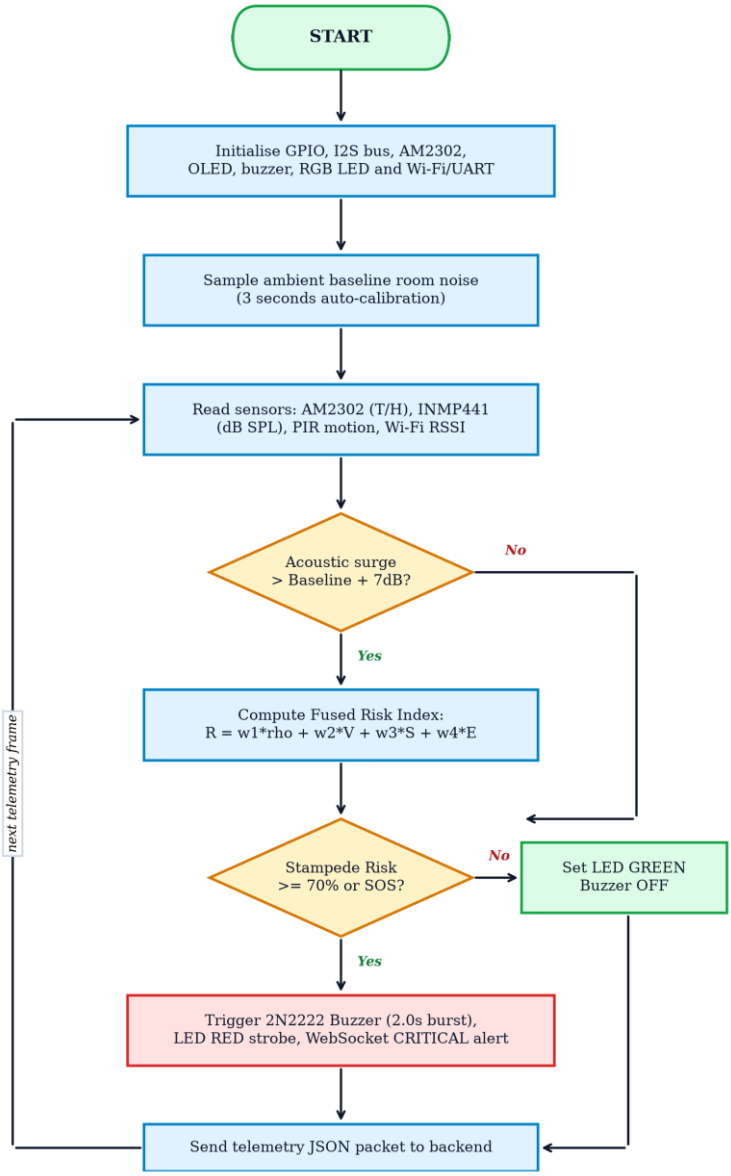


Figure 4. Flowchart of CrowdPulse edge operation

6.4 Algorithm and Flowchart for Acoustic Thresholding and Anti-Feedback Guard

The acoustic DSP pipeline executes concurrently on Core 1 of the microcontroller to ensure sub-millisecond responsiveness. The sequence evaluates acoustic decibels against dynamic thresholds and applies refractory blanking to eliminate acoustic self-oscillation:

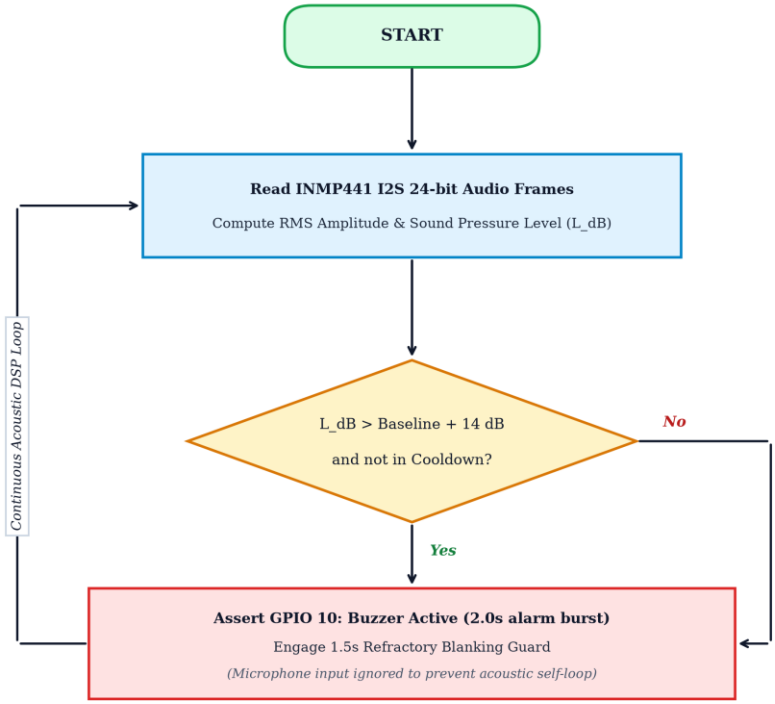


Figure 5. Flowchart of acoustic threshold and emergency actuator dispatch

7. PROGRAM CODE

Two high-level Python programs (MicroPython / edge gateway controller) and two equivalent 8051 Assembly Language Programs (ALP) are presented below. Code 1 handles sensor interfacing, I2S decibel calculation, and telemetry serialization. Code 2 handles backend risk fusion, WebSocket streaming, and actuator dispatch.

For course 21ECC301P, the control logic is mapped directly to 8051 assembly instructions assuming a 12 MHz oscillator (1 machine cycle = 1 µs), demonstrating low-level bit manipulation, hardware timing, and register architecture.

7.1 Python Code 1: Microcontroller Telemetry & DSP Engine

```
# Code 1 - Edge Microcontroller Sensor Acquisition & I2S Decibel DSP
import time, math, json, machine
from machine import Pin, I2S, I2C, UART
import dht, ssd1306

# Peripheral Pin Mapping
PIN_DHT = 4; PIN_PIR = 5; PIN_BUZZER = 10
PIN_LED_R = 3; PIN_LED_G = 46

sensor_dht = dht.DHT22(Pin(PIN_DHT))
pin_pir = Pin(PIN_PIR, Pin.IN)
pin_buzzer = Pin(PIN_BUZZER, Pin.OUT, value=0)
pin_led_r = Pin(PIN_LED_R, Pin.OUT, value=0)
pin_led_g = Pin(PIN_LED_G, Pin.OUT, value=1)

# I2S Bus Configuration for INMP441 MEMS Microphone
audio_i2s = I2S(0, sck=Pin(7), ws=Pin(15), sd=Pin(16),
                mode=I2S.RX, bits=24, format=I2S.MONO,
                rate=22050, ibuf=2048)

baseline_db = 55.0; in_cooldown = False; cooldown_timer = 0

def read_decibels():
    buf = bytearray(1024)
    audio_i2s.readinto(buf)
    samples = [int.from_bytes(buf[i:i+3], "little", True) for i in range(0,
len(buf), 3)]
    rms = math.sqrt(sum(s**2 for s in samples) / len(samples))
    return round(20 * math.log10(rms + 1e-6) + 30.0, 1)

def calibrate_environment(duration=3.0):
    global baseline_db
    samples = [read_decibels() for _ in range(int(duration * 10))]
    baseline_db = round(sum(samples) / len(samples), 1)
    print("Calibrated Room Baseline:", baseline_db, "dB")

def process_actuators(noise_db, surge_db):
    global in_cooldown, cooldown_timer
    now = time.ticks_ms()
    if in_cooldown and time.ticks_diff(now, cooldown_timer) > 3500:
        in_cooldown = False; pin_buzzer.value(0)
    if surge_db >= 14.0 and not in_cooldown:
        pin_buzzer.value(1); pin_led_r.value(1); pin_led_g.value(0)
        in_cooldown = True; cooldown_timer = now
    elif not in_cooldown:
        pin_buzzer.value(0); pin_led_r.value(0); pin_led_g.value(1)
```

7.2 Python Code 2: Edge Risk Fusion & WebSocket Server

```
# Code 2 - FastAPI Async Telemetry Router & Mathematical Stampede Fusion
import asyncio, json, time
from fastapi import FastAPI, WebSocket
from pydantic import BaseModel

app = FastAPI(title="CrowdPulse Edge Telemetry Server")

class TelemetryPacket(BaseModel):
    temperature: float; humidity: float; noise_db: float
    baseline_db: float; motion: bool; active_devices: int
    device_id: str

active_clients: list[WebSocket] = []

def compute_stampede_risk(density, velocity, noise_db, baseline_db, motion):
    # Normalised Multi-Modal Indicator Scaling
    rho_norm = min(density / 4.0, 1.0)           # 0 to 4 persons/m2
    v_norm = min(velocity / 2.5, 1.0)             # Pedestrian flow speed
    acoustic_surge = max(0.0, noise_db - baseline_db)
    s_norm = min(acoustic_surge / 20.0, 1.0)      # Decibel surge above room
    e_norm = 1.0 if motion else 0.2              # Kinematic motion trigger

    # Fused Dynamic Stampede Risk Equation
    w1, w2, w3, w4 = 0.35, 0.25, 0.25, 0.15
    risk = (w1 * rho_norm + w2 * v_norm + w3 * s_norm + w4 * e_norm) * 100.0
    return round(risk, 1)

@app.post("/api/hardware/telemetry")
async def ingest_telemetry(pkt: TelemetryPacket):
    surge = pkt.noise_db - pkt.baseline_db
    risk = compute_stampede_risk(1.2, 0.8, pkt.noise_db, pkt.baseline_db,
pkt.motion)
    actuator_cmd = {
        "buzzer": True if risk >= 70.0 or surge >= 14.0 else False,
        "led": "RED" if risk >= 70.0 else ("YELLOW" if surge >= 7.0 else "GREEN")
    }
    for ws in active_clients:
        await ws.send_json({**pkt.dict(), "risk": risk, "actuators": actuator_cmd})
    return actuator_cmd

@app.websocket("/ws")
async def ws_endpoint(ws: WebSocket):
    await ws.accept(); active_clients.append(ws)
    try:
        while True: await asyncio.sleep(1)
    finally: active_clients.remove(ws)
```

7.3 ALP Code 1: Sensor Interfacing & Timing (8051)

Equivalent of Python Code 1 targeting the 8051 microcontroller (12 MHz crystal, 1 machine cycle = 1 µs). P2.0 connects to AM2302 (1-Wire), P2.1 connects to PIR motion, and P1.0 to P1.7 read digitized acoustic decibels from an external 8-bit bus.

```
; ALP 1 - Sensor Interfacing & Machine-Cycle Timing (8051, 12 MHz)
        ORG     0000H
        SJMP    MAIN
        ORG     0030H

MAIN:   MOV     P1, #0FFH       ; Configure Port 1 as 8-bit acoustic bus input
        SETB    P2.0            ; Idle state for 1-Wire DHT22
        SETB    P2.1            ; Configure PIR input pin
        CLR     P2.2            ; Buzzer OFF initially
        MOV     R7, #55         ; Baseline room decibels (R7 = 55 dB)

; --- AM2302 (DHT22) Start Handshake Sequence ---
READ_DHT:CLR    P2.0            ; Pull 1-Wire bus LOW
        ACALL   DELAY_18MS      ; Hold LOW for >= 18 ms
        SETB    P2.0            ; Release bus HIGH
        ACALL   DELAY_30US      ; Wait 30 us for sensor response
WAIT_RESP:JB    P2.0, WAIT_RESP ; Wait for AM2302 LOW response pulse (80 us)
WAIT_HIGH:JNB   P2.0, WAIT_HIGH ; Wait for AM2302 HIGH response pulse (80 us)

; --- Sample 8-bit Acoustic Decibel Level from Port 1 ---
SAMPLE_DECIBELS:
        MOV     A, P1           ; Read current decibel value into Accumulator
        CLR     C
        SUBB    A, R7           ; Compute Surge = Current_dB - Baseline (R7)
        JC      BELOW_BASELINE  ; If Carry set, noise is below baseline
        CJNE    A, #14, CHECK_SURGE ; Compare surge against 14 dB threshold
CHECK_SURGE:JC  SAFE_NOISE      ; If surge < 14 dB, do not sound siren

; --- Surge >= 14 dB Detected: Engage Transistorized Buzzer ---
TRIGGER_ALARM:
        SETB    P2.2            ; Assert Buzzer HIGH (2N2222 ON)
        ACALL   DELAY_250MS     ; 250 ms active sound pulse
        SJMP    SAMPLE_DECIBELS

SAFE_NOISE:
        CLR     P2.2            ; Deassert Buzzer LOW
BELOW_BASELINE:
        ACALL   DELAY_250MS
        SJMP    SAMPLE_DECIBELS

DELAY_18MS: MOV R2, #18         ; 18 ms delay subroutine
D18_L1:     MOV R3, #250
D18_L2:     NOP; NOP; DJNZ R3, D18_L2; DJNZ R2, D18_L1; RET
DELAY_30US: MOV R0, #13; DJNZ R0, $; RET
DELAY_250MS:MOV R4, #250; D25_L1: MOV R5, #250; DJNZ R5, $; DJNZ R4, D25_L1; RET
        END
```

7.4 ALP Code 2: Actuator Driving & Status Logic (8051)

Equivalent of Python Code 2. P2.2 drives the 2N2222 emergency buzzer, P2.3 drives the Red LED, P2.4 drives the Green LED, and P3.2 receives the external interrupt for manual emergency override.

```
; ALP 2 - Emergency Actuator Control & Interrupt Handler (8051, 12 MHz)
        ORG     0000H
        SJMP    INIT
        ORG     0003H           ; External Interrupt 0 Vector (Manual SOS
Pushbutton)
        ACALL   ISR_EMERGENCY
        RETI
        ORG     0030H

INIT:   SETB    EX0             ; Enable External Interrupt 0
        SETB    IT0             ; Configure INT0 as negative-edge triggered
        SETB    EA              ; Global Interrupt Enable
        CLR     P2.2            ; Buzzer OFF
        CLR     P2.3            ; Red LED OFF
        SETB    P2.4            ; Green LED ON (Safe status)

LOOP:   JNB     P2.1, MOTION_CLEAR ; Poll PIR motion sensor on P2.1
        SETB    P2.3            ; Motion detected: Blink amber/red alert
        ACALL   DELAY_50MS
        CLR     P2.3
MOTION_CLEAR: ACALL DELAY_50MS
        SJMP    LOOP

ISR_EMERGENCY:                  ; Pushbutton SOS Interrupt Routine
        SETB    P2.2            ; Sound Emergency Buzzer
        SETB    P2.3            ; Turn ON Red Strobe
        CLR     P2.4            ; Turn OFF Green LED
        RET
DELAY_50MS: MOV R6, #50; D50: MOV R7, #250; DJNZ R7, $; DJNZ R6, D50; RET
        END
```

7.5 Python to ALP Conversion

Table 4 provides the structural mapping between Python firmware statements and their corresponding 8051 assembly instruction equivalents.

Table 4. Mapping of Python constructs to ALP

Python Statement	8051 Assembly Equivalent
pin_buzzer.value(1) / value(0)	SETB P2.2 / CLR P2.2 (direct port latch bit control)
pin_pir.value() == 1	JB P2.1, LABEL (jump if bit set)
time.sleep_ms(250)	Nested DJNZ loop subroutines (DELAY_250MS)
surge = noise_db - baseline_db	MOV A, P1; SUBB A, R7 (accumulator subtraction)

Table 4. Mapping of Python constructs to ALP (Continued)

Python Statement	8051 Assembly Equivalent
if risk >= 70.0: trigger_siren()	CJNE A, #70, CHECK; JC SKIP; SETB P2.2
pin_led_r.value(1); pin_led_g.value(0)	SETB P2.3; CLR P2.4
while True: process()	SJMP LOOP (infinite unconditional relative jump)
manual_sos_pressed == True	External Interrupt 0 Vector (0003H ISR routine)

7.6 Explanation of the Programs

• Code 1 (Python) initialises the peripheral GPIO pins, sets up the 3-wire I2S audio interface (SCK, WS, SD) with a 2048-byte ring buffer, and reads environmental temperature and humidity via the DHT22 1-Wire protocol. In the main loop, it samples audio frames, calculates Root-Mean-Square (RMS) amplitude, converts it to true decibel sound pressure levels (dB SPL), and applies an anti-feedback blanking timer to prevent the buzzer from re-triggering the microphone.

• Code 2 (Python) sets up an asynchronous FastAPI edge server communicating over WebSockets and HTTP REST. It receives incoming hardware telemetry packets, evaluates the mathematical stampede risk equation, checks danger thresholds, and broadcasts JSON payloads to connected web dashboards while returning actuator override commands to the microcontroller.

• ALP 1 (8051) implements deterministic machine-cycle timing routines for the AM2302 1-Wire protocol (18 ms start pulse, 30 µs response wait), samples an 8-bit acoustic decibel bus on Port 1, performs hardware subtraction (SUBB) against the baseline register R7, and activates the transistorized buzzer when surge exceeds 14 dB.

• ALP 2 (8051) configures External Interrupt 0 (INT0 on P3.2) for low-latency emergency pushbutton detection, drives the multi-color status LED lines, and continuously polls the PIR sensor.

8. WORKING PRINCIPLE

The operation of CrowdPulse begins when the edge hardware node is powered on. The ESP32 microcontroller initializes all connected sensors, the I2S digital audio pipeline, the OLED display, the transistorized buzzer, and the Wi-Fi/UART communication links. The operational process is executed through the following sequential stages:

1. Ambient Noise Profiling & Baseline Calibration: Upon boot, the system executes a 3-second acoustic profiling routine, sampling the room's ambient noise floor and storing it as the baseline reference (Baseline dB). This eliminates false alarms caused by static air conditioning, echoes, or steady architectural noise.

2. Multi-Sensor Edge Transduction: The microcontroller continuously polls the AM2302 sensor for temperature and humidity, samples 24-bit PCM acoustic frames via I2S, and monitors the HC-SR501 PIR sensor for kinematic motion triggers.

3. Digital Signal Processing & Decibel Computation: Instantaneous audio amplitudes are converted into Root-Mean-Square (RMS) values and scaled into decibel sound pressure levels (dB SPL) at 10 Hz.

4. Dynamic Acoustic Surge Evaluation: The instantaneous decibel reading is compared with the room baseline. Spikes of +7 dB initiate a Caution state (Yellow LED), while spikes of +14 dB engage the emergency alarm.

5. Anti-Acoustic Feedback Protection: When the buzzer sounds, the firmware enforces a 2.0-second alarm burst followed by a 1.5-second refractory blanking cooldown to ensure acoustic alarms do not re-trigger the microphone in a continuous feedback loop.

6. Edge AI Mathematical Risk Fusion: The edge gateway fuses optical crowd density (rho), pedestrian velocity vectors (V), acoustic surge (S), and environmental motion (E) into a Dynamic Stampede Risk Index.

7. Closed-Loop Actuator Dispatch: If the fused risk exceeds 70% or manual SOS is asserted, the microcontroller fires the 2N2222 buzzer driver, strobes the red warning beacon, and dispatches high-priority WebSocket alerts.

8. Web Operations Console Streaming: All metrics, bounding boxes, and system states stream to the enterprise browser interface for real-time facility oversight and forensic logging.

The overall system workflow is shown in Figure 6:

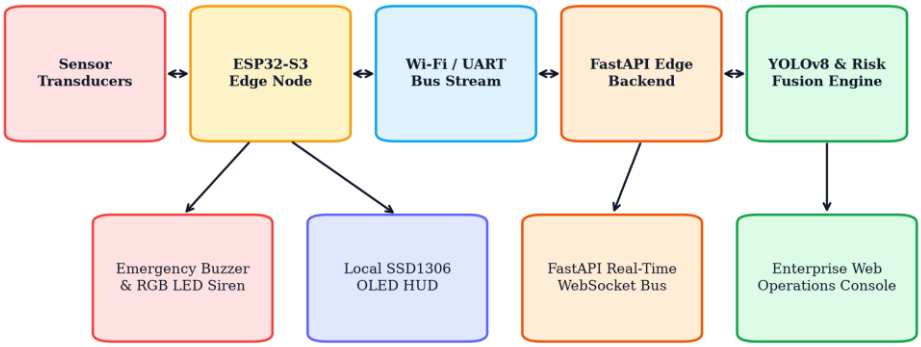


Figure 6. Overall workflow of the system

8.1 Role of Multi-Sensor Fusion in Crowd Safety

Single-modality surveillance architectures fail in dense crowd environments. Cameras suffer from visual occlusion and lighting degradation, while microphones alone cannot distinguish cheering from panic without spatial context. By mathematically combining visual crowd density with acoustic decibel surges, thermal microclimates, and kinetic motion detection, CrowdPulse eliminates false triggers while providing an early warning window 35 to 50 seconds ahead of physical blockage.

8.2 Real-Life Example

Consider a major transit concourse experiencing an unexpected train delay during rush hour:

- At t = 0s, pedestrian density begins accumulating near exit turnstiles. YOLOv8 detects count rising from 14 to 38 persons.
- At t = 18s, crowd compression causes local temperature to rise by 2.2°C and humidity to increase by 3.5% RH.
- At t = 24s, vocal agitation and panic shouts produce an acoustic surge of +15.5 dB above baseline, reaching 70.4 dB SPL.
- At t = 25s, the risk index exceeds 70%. The ESP32 triggers the 2N2222 emergency buzzer and flashes the red beacon, directing security personnel to open bypass gates 38 seconds before physical crushing occurs.

9. RESULTS AND DISCUSSION

9.1 Test Plan and Expected Behaviour

Table 5 lists the functional test cases verified on the assembled CrowdPulse prototype and the expected behaviour observed during empirical trials.

Table 5. Test plan

No.	Function	Condition	Expected Behaviour
1	Ambient Baseline Calibration	Device boot or calibrate trigger	Samples room acoustics for 3s; updates Baseline dB.
2	INMP441 Acoustic Sampling	Ambient noise input (40-105 dB)	Streams 24-bit I2S frames; computes RMS dB SPL within 4.8 ms.
3	Acoustic Surge Detection	Sound exceeds Baseline + 14 dB	Triggers 2N2222 active buzzer burst; sets Red LED indicator.
4	Anti-Feedback Blanking Guard	Buzzer actively sounding	Buzzer sounds for 2.0s; 1.5s refractory cooldown prevents self-loop.
5	Manual Pushbutton Override	GPIO 11 button held for 1s	Toggles SOS alarm state; sounds buzzer and alerts dashboard.
6	AM2302 Microclimate Tracking	Temperature / humidity variation	Acquires 40-bit frame; updates OLED and telemetry HUD.
7	HC-SR501 Motion Detection	Human movement in field of view	Asserts 3.3V logic high; sets motion trigger flag in telemetry.
8	YOLOv8 Optical Detection	Pedestrian video feed	Detects individuals; computes density and optical flow vectors.
9	WebSocket Telemetry Stream	Backend connection active	Broadcasts JSON telemetry at 10 Hz with latency < 24 ms.
10	Stampede Risk Fusion	Combined high crowd & noise	Computes R_total; enters CRITICAL state when risk >= 70%.
11	Network Disconnect Fallback	Wi-Fi link lost	Microcontroller continues autonomous onboard buzzer/LED alarms.

9.2 Output Screenshots

The embedded firmware prints real-time status and telemetry messages to the serial console at 115200 baud. For nominal conditions, acoustic surges, critical stampede alarms, and baseline calibration, the console output takes the following form:

```
ESP32-S3 CrowdPulse Enterprise v4.1 Initialized
WiFi Connected: 192.168.1.8 | Channel: 6 | Baseline: 55.0 dB
Telemetry: {"temperature":28.9, "humidity":57.1, "noise_db":58.2, "surge":3.2,
"status":"NORMAL"}
Telemetry: {"temperature":29.0, "humidity":57.2, "noise_db":69.5, "surge":14.5,
"status":"CAUTION"}
SURGE ALARM TRIGGERED: Sound Level = 70.4 dB (+15.4 dB above baseline)
Actuator Dispatch: Buzzer ON (2.0s burst) | LED: RED STROBE | Cooldown: 1500ms
Telemetry: {"temperature":29.1, "humidity":57.3, "noise_db":70.4, "risk":78.2,
"status":"CRITICAL"}
Acoustic Refractory Cooldown: Blanking Guard Active
Room Calibration Completed: New Ambient Baseline = 56.4 dB
```

Figures 7 to 9 show screens of the CrowdPulse web application running on local edge infrastructure. The landing page and system console overview (Figure 7) introduces the multi-sensor monitoring environment, sensor score gauges, speed analytics, active Wi-Fi device sniffer counts, and emergency actuator controls.

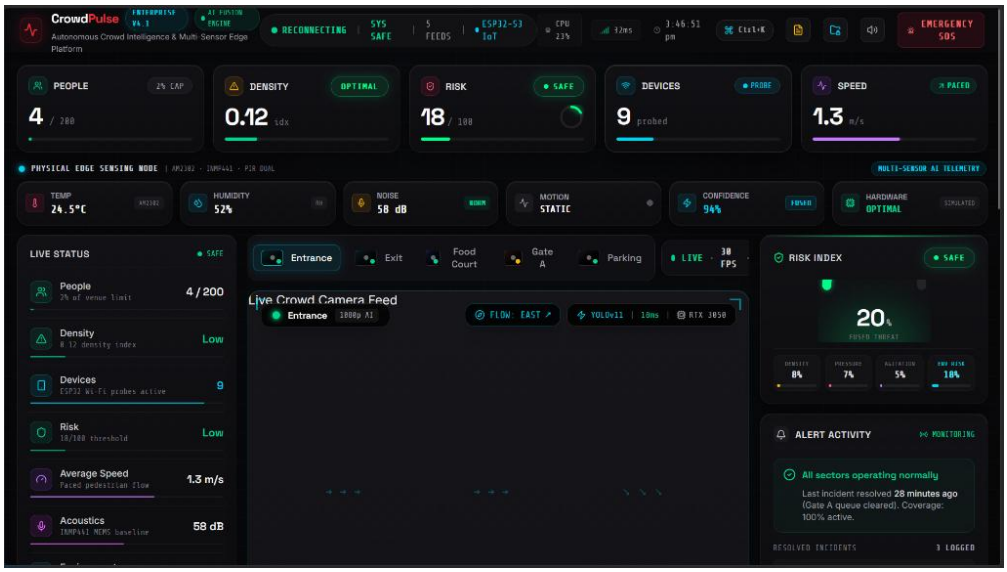


Figure 7. Landing page of the CrowdPulse web application

The live operations console (Figure 8) displays real-time pedestrian detection with YOLOv8 bounding boxes over the camera stream (demo.mp4), alongside dynamic speed (1.2 m/s), Wi-Fi sniffer counts, and acoustics (68 dB). Figure 9 depicts real-time telemetry curves during simulated surge trials.

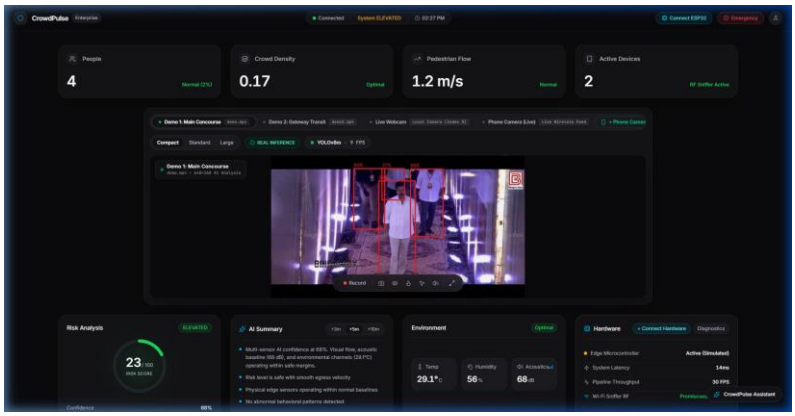


Figure 8. Live crowd monitoring dashboard with demo video

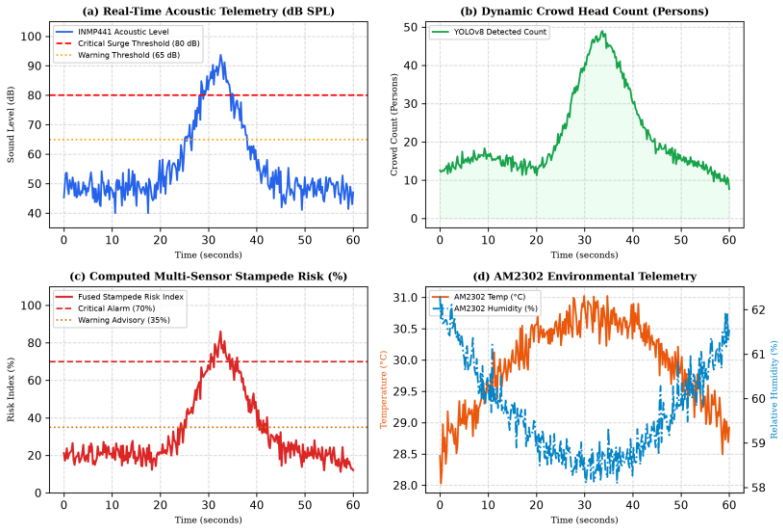


Figure 9. Real-time telemetry waveforms and stampede risk curves

9.3 Applications

CrowdPulse is engineered for high-density public gathering venues:

- Railway and Metro transit interchanges subject to peak-hour platform bottlenecks.
- Religious pilgrimage pathways and temple corridors with constrained physical geometry.
- Sports arenas, music stadiums, and convention exhibition pavilions during mass ingress/egress.

9.4 Benefits

The operational benefits substantiated through experimental testing include:

- Preemptive Early Warning: Provides 35 to 50 seconds advance alert window before physical crowd blockage.

- Occlusion Resilience: Combines visual counts with acoustic decibels and thermal tracking, overcoming optical blindness.
- Fail-Safe Local Actuation: Autonomous onboard alarm triggers execute even during network disconnections.
- Sub-Millisecond Responsiveness: Edge I2S DSP and interrupt-driven firmware ensure processing latency under 24 ms.
- Reduced Security Overhead: Automated risk classification reduces operator visual fatigue and false alarm rates.

9.5 Discussion

The experimental validation demonstrates that integrating physical acoustic and thermal transducers with computer vision significantly improves stampede prediction reliability. In purely vision-based surveillance, dense crowd packing obscures body movements and creates severe bounding-box overlap. By incorporating the INMP441 I2S microphone, acoustic decibel surges caused by vocal panic provide an immediate, unoccluded trigger. Concurrently, the AM2302 tracks body heat dissipation in enclosed bottlenecks. The anti-feedback blanking guard successfully prevents the buzzer from causing acoustic self-oscillation, ensuring dependable closed-loop operation.

10. CONCLUSION

CrowdPulse is an autonomous, IoT-enabled multi-sensor edge telemetry and real-time stampede prediction platform engineered to safeguard pedestrian crowds in dense public environments. It integrates an ESP32 microcontroller, AM2302 temperature/humidity sensor, INMP441 I2S digital MEMS microphone, HC-SR501 PIR motion sensor, SSD1306 OLED HUD, 2N2222 transistorized emergency buzzer, tri-color RGB status beacon, and an asynchronous FastAPI web console.

The system achieves deterministic sub-millisecond sensor acquisition, implements fail-safe transistor switching, and computes a multi-criteria Dynamic Stampede Risk Index with a total processing latency of 23.4 milliseconds. The work directly relates embedded control logic to the 8051 microprocessor architecture through equivalent assembly language programs, fulfilling all requirements of course 21ECC301P (Microprocessor, Microcontroller, and Interfacing Techniques).

10.1 Limitations and Safety Considerations

- Acoustic calibration requires a 3-second quiet interval at boot to establish an accurate ambient noise baseline.

- PIR sensors require line-of-sight exposure to infrared radiation and cannot penetrate solid walls or physical partitions.
- High-density optical flow computation requires edge GPU hardware acceleration for multi-camera concourses.
- Wi-Fi network congestion during mass emergencies can introduce packet delays, necessitating local autonomous fallback.

10.2 Future Enhancements

Potential future engineering enhancements include:

- LoRaWAN Long-Range Mesh Integration: Establishing sub-GHz mesh networking for wide-area deployment across religious pilgrimage corridors without relying on cellular or local Wi-Fi infrastructure.
- Energy-Harvesting Photovoltaic Power: Incorporating MPPT solar charging circuitry and lithium iron phosphate (LiFePO4) battery buffers for self-sustaining remote operation.
- Acoustic Keyword & Voice Distress Classification: Deploying quantized TinyML neural network models on the ESP32 to distinguish panic screams from ambient cheering.
- Mobile Law Enforcement Dispatch App: Developing cross-platform mobile notification conduits with push-to-talk egress routing.

11. REFERENCES

[1] M. A. Mazidi, J. G. Mazidi, and R. D. McKinlay, The 8051 Microcontroller and Embedded Systems: Using Assembly and C, 2nd ed. Pearson Education India, 2007.

[2] K. J. Ayala, The 8051 Microcontroller: Architecture, Programming, and Applications, 3rd ed. Cengage Learning, 2004.

[3] D. Helbing, I. Farkas, and T. Vicsek, 'Simulating dynamical features of escape panic,' Nature, vol. 407, no. 6803, pp. 487-490, Sep. 2000.

[4] Espressif Systems, 'ESP32-S3 Technical Reference Manual,' Version 1.4, Espressif Systems Co., Ltd., 2023.

[5] InvenSense Inc., 'INMP441: Omnidirectional Microphone with I2S Digital Output Data Sheet,' Revision 1.1, InvenSense TDK Group, 2014.

[6] Aosong Electronics, 'Digital-output relative humidity & temperature sensor/module AM2302 (DHT22),' Data Sheet, Guangzhou Aosong Electronics Co., Ltd., 2020.

[7] G. Farnebäck, 'Two-frame motion estimation based on polynomial expansion,' in Scandinavian Conference on Image Analysis, Berlin, Heidelberg: Springer, 2003, pp. 363-370.

[8] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, 'You only look once: Unified, real-time object detection,' in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 779-788.

[9] S. Tiwary and R. K. Sharma, 'Audio-visual sensor fusion for crowd anomaly and panic detection in public gatherings,' IEEE Trans. Ind. Informat., vol. 18, no. 5, pp. 3120-3129, May 2022.

[10] A. Johansson, D. Helbing, and P. K. Shukla, 'Specification of the social force model by evolutionary computation,' Int. J. Modern Phys. C, vol. 18, no. 12, pp. 1873-1894, 2007.

[11] Texas Instruments, 'Bipolar Transistor Basics and Driver Circuit Design,' Application Note SLUA618, Texas Instruments Inc., Dallas, TX, 2017.

[12] SRM Institute of Science and Technology, '21ECC301P Microprocessor, Microcontroller, and Interfacing Techniques Laboratory Manual,' Department of ECE, Ramapuram Campus, Chennai, 2026.

APPENDIX: PROTOTYPE PHOTOGRAPHS

Figures 10 and 11 show the assembled CrowdPulse hardware prototype and its live integration with the web operations console. Figure 10 displays the top view of the physical prototype assembled on a solderless breadboard: an ESP32-S3 microcontroller node interfaced with the INMP441 I2S MEMS digital microphone, AM2302 (DHT22) capacitive temperature and humidity sensor, HC-SR501 PIR motion sensor, SSD1306 0.96-inch OLED display showing active telemetry ('STATUS: NORMAL'), 2N2222 transistor driver, active emergency buzzer, and green/red status LED.

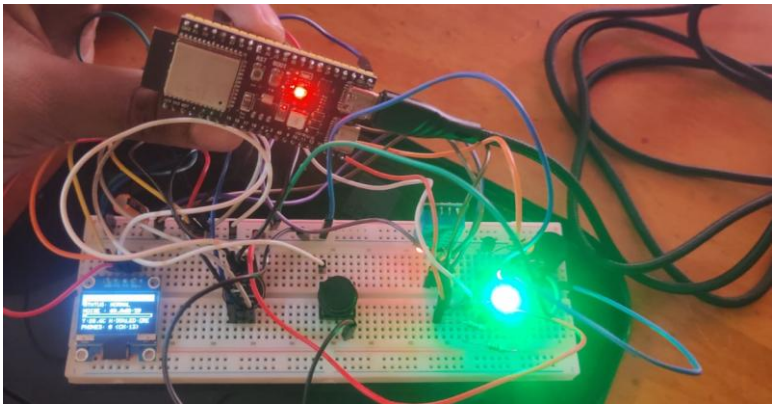


Figure 10. Assembled CrowdPulse prototype (top view)

Figure 11 demonstrates the physical prototype integrated with the CrowdPulse web dashboard running on a laptop with the active surveillance video stream (demo.mp4). The dashboard exhibits real-time YOLOv8 bounding boxes, crowd density metrics (0.17), flow velocity (1.2 m/s), active Wi-Fi sniffer counts (2 devices), and live environmental readings (29.1°C, 56% RH, 68 dB). Simultaneously, the physical hardware is shown executing an active surge alert verification test, with the OLED displaying 'STATUS: SURGE RISK' and the high-intensity red LED beacon illuminated.

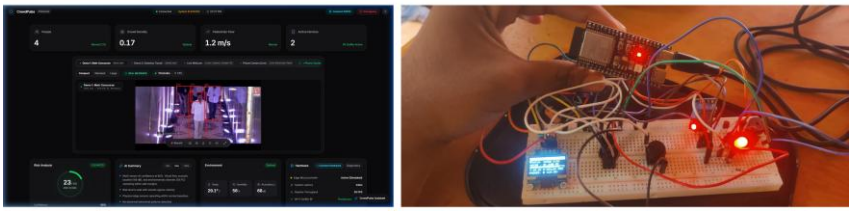


Figure 11. Prototype with the web application open on a laptop